



Model Context Protocol (MCP)

AICB-P2T3 · Ngày 26 · Chương 6 — Chuẩn hóa Tool Integration

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 6

HÃY SUY NGHĨ...

“MCP giải quyết vấn đề gì mà function calling thông thường không giải quyết được?”

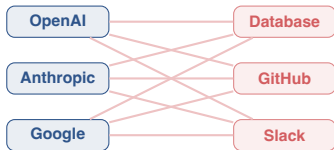
Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Vấn đề $N \times M$ và tại sao cần MCP
2. MCP Architecture
3. Build MCP Server với Python SDK
4. Security, Registry & Versioning
5. Demo & Thực hành
6. MCP trong thực tế 2026
7. Real Use Cases & Useful Servers

Vấn đề $N \times M$ và tại sao cần MCP

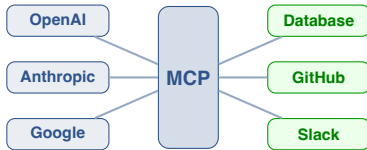
Mỗi provider một format riêng — integration nightmare

Trước MCP: $N \times M$ connections



$3 \times 3 = 9$ adapters

Sau MCP: $N+M$ connections



$3 + 3 = 6$ adapters

MCP giống **USB-C** — một chuẩn kết nối cho mọi thiết bị, thay vì mỗi hãng một cổng riêng. Write once, run anywhere.

97M+

Monthly SDK
downloads

10,000+

Community
MCP servers

LF

Linux Foundation
donation

MCP không chỉ là Anthropic — đã được Google, Microsoft, OpenAI adopt. **A2A protocol** (Agent-to-Agent) complement MCP cho agent-to-agent communication.

Function Calling (truyền thống)

- Tightly coupled to provider
- OpenAI format $\neq$ Anthropic $\neq$ Google
- Chỉ hỗ trợ **tool calls**
- Viết lại adapter mỗi khi đổi provider

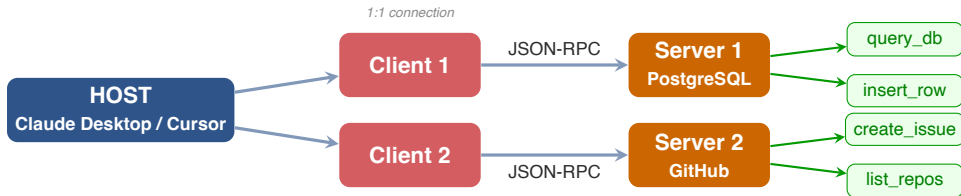
MCP (chuẩn mới)

- Universal, provider-agnostic
- JSON-RPC 2.0 standard
- Tools + Resources + Prompts + Sampling + Elicitation
- Write tool once, any client dùng được

MCP Architecture

Host → Client → Server → Tools

02



Host **trusts** Client (same process)
Client **verifies** Server (auth)
Server **validates** all inputs

stdio: same machine, low latency
SSE/HTTP: remote, production
Cùng protocol, khác transport

Primitive	Vai trò	Ai kiểm soát?	Ví dụ
Tools	Callable functions	LLM quyết định gọi	<code>query_db()</code> , <code>send_email()</code>
Resources	Read-only data (URI)	App cung cấp context	<code>file://docs/guide.md</code>
Prompts	Reusable templates	User chọn	<code>summarize-code</code> template
Sampling	LLM completions	Server yêu cầu	Server gọi LLM qua host
Elicitation	Structured input	Server hỏi user	Form input qua host UI

Lưu ý: Hầu hết team chỉ dùng Tools — nhưng **Resources** và **Prompts** underutilized. Dùng Resources cho dynamic context injection thay vì hardcode vào system prompt.



LLM chọn tool dựa trên **name + description**. Mô tả rõ ràng, cụ thể = LLM chọn đúng tool. Mô tả mơ hồ = agent gọi sai tool → failure.

Build MCP Server với Python SDK

FastMCP — decorator-based, production-ready

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("sales-db")

@mcp.tool()
async def query_sales(
    region: str, quarter: str
) -> dict:
    """Query sales data by region
    and quarter. Returns revenue,
    units_sold, top_products."""
    result = await db.query(
        region=region, quarter=quarter
    )
    return result.to_dict()

@mcp.resource("sales://schema")
async def get_schema() -> str:
    return db.get_schema_ddl()
```

3 decorators:

1. `@mcp.tool()` — callable function
2. `@mcp.resource()` — read-only data
3. `@mcp.prompt()` — reusable template

Lưu ý: Tool description (doc-string) là **quan trọng nhất**. LLM đọc description để quyết định gọi tool nào.

Error handling: Return structured errors, **không raise exceptions** — client cần error message rõ ràng để retry/fallback.

```
# claude_desktop_config.json
config = {
  "mcpServers": {
    "sales-db": {
      "command": "python",
      "args": ["-m", "sales_mcp"],
      "env": {
        "DB_URL": "postgresql://..."
      }
    },
    "github": {
      "command": "npx",
      "args": [
        "@modelcontextprotocol/"
        "server-github"
      ]
    }
  }
}
```

Workflow chuẩn:

1. **Write** MCP server (Python/TS)
2. **Test** với MCP Inspector
3. **Config**
claude_desktop_config.json
4. **E2E test** trong Claude Desktop

Web UI test tools **without LLM** —
verify schemas, output format, error
handling trước khi integrate.

Debugging: Inspector first → Claude Desktop logs →
LangSmith traces

Security, Registry & Versioning

Production MCP cần auth, discovery, và backward compatibility

Transport Security: OAuth 2.0, TLS

Input Validation: server-side

Permission Scope: least privilege

Audit Log

4 tầng bảo mật:

1. **Transport:** OAuth 2.0 cho SSE/HTTP, TLS encrypt
2. **Validation:** validate *tất cả* inputs server-side
3. **Permissions:** mỗi tool chỉ có quyền cần thiết
4. **Audit:** log mọi tool call + kết quả

Lưu ý: stdio transport (local) không cần OAuth — nhưng SSE/HTTP (production remote) **bắt buộc** phải có auth. Đừng expose MCP server ra internet không có auth!



Tool Registry — Central catalog
— agent discovers tools at runtime
matching task requirements

Versioning rules:

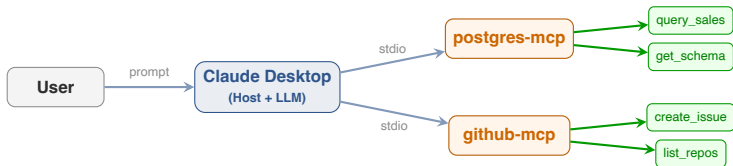
- **Semver**: major.minor.patch
- Breaking changes = **major** bump
- Deprecation: warn period trước khi remove old versions
- Clients cần migration time

Demo & Thực hành

Live demo + Lab: build MCP server từ scratch

05

1. **Setup:** Claude Desktop + custom PostgreSQL MCP server + official GitHub MCP server
2. **Inspector test:** verify tool schemas, output format, error handling
3. **Task:** “Query sales DB rồi create GitHub issue with analysis” — requires cả 2 servers
4. **E2E:** Claude routes `query_db` sang `postgres-mcp`, `create_issue` sang `github-mcp` seamlessly



User hỏi: “phân tích doanh số Q3 rồi tạo issue” → Claude tự route `query_sales` sang `postgres-mcp`, nhận data, phân tích, rồi gọi `create_issue` sang `github-mcp`.
Seamless multi-tool orchestration.

Mục tiêu: Build custom MCP server + test cross-client compatibility

Deliverable: GitHub repo: MCP server (3 tools) + Inspector test screenshots + Claude Desktop integration demo

Thời gian: 2 giờ

1. **Build MCP Server:** FastMCP, expose 3 tools (search, insert, aggregate) cho database
2. **Add Resource:** expose database schema qua `@mcp.resource()` cho dynamic context
3. **Test với Inspector:** verify tool schemas, test calls, check error responses
4. **Claude Desktop:** config json, E2E test, verify multi-tool routing

GitHub repo + README: tool descriptions, Inspector screenshots, Claude Desktop demo video (2 phút). **Bonus:** implement auth cho SSE transport.

MCP trong thực tế 2026

Từ protocol mới sang ecosystem production-grade

06

10,000+

Active public
MCP servers

97M+

Monthly SDK
downloads

AAIF

Stewardship under
Linux Foundation

MCP đã đi từ “Anthropic protocol” sang **vendor-neutral standard**: ChatGPT, Cursor, Gemini, Microsoft Copilot, VS Code đều đã xuất hiện trong ecosystem adopt/support.

- **Transport guidance rõ hơn:** remote production đang nghiêng về `streamable-http`; SSE dần trở thành legacy/deprecated path ở nhiều client
- **Capability negotiation** trở thành tư duy cốt lõi: client và server khai báo rõ hỗ trợ gì trước khi dùng
- **Không chỉ tools:** resources, prompts, roots, sampling, elicitation đều đã có vai trò thực tế hơn
- **Scale problem xuất hiện thật:** khi một host có hàng trăm tools, context window và latency trở thành bottleneck
- **Governance trưởng thành:** MCP được đưa vào Agentic AI Foundation để tránh lock-in theo một vendor

Lưu ý: Nếu deck/team của bạn vẫn nghĩ MCP chỉ là “function calling nhưng tên mới”, thì đang bỏ lỡ phần quan trọng nhất: **tool discovery, context control, và multi-client interoperability.**

1. **Tools**: model thực hiện action hoặc query
2. **Resources**: host đọc data/context theo URI
3. **Prompts**: reusable interaction templates
4. **Roots**: client chia sẻ workspace roots an toàn cho server
5. **Sampling**: server yêu cầu host/LLM suy luận thêm
6. **Elicitation**: server xin thêm thông tin từ user qua host

Ba primitive này trả lời câu hỏi: server **có thể cung cấp gì** cho host?

Ba primitive này trả lời câu hỏi: server **cần host hỗ trợ gì** để hoàn tất workflow?

Design takeaway: MCP là protocol cho **context exchange**, không chỉ protocol cho “tool call”.

Real Use Cases & Useful Servers

MCP có giá trị nhất khi ghép context, action và policy lại với nhau

07

Server	Loại	Dùng để làm gì	Khi nào nên dùng
GitHub MCP	official	repo, issue, PR, code search	Agent code theo ticket-to-PR loop
Sentry MCP	official	errors, stack trace, release regression	Debug production issue rồi mở fix/issue ngay
OpenAI Docs MCP	official	docs search + page fetch	Codex/Cursor cần đúng API/schema mới nhất
Context7 MCP	de-facto standard	version-specific library docs	Code với framework/library thay đổi nhanh
Playwright MCP	official	browser automation, repro UI, E2E checks	Tái hiện bug UI và verify fix end-to-end
Slack / Notion MCP	official SaaS	team chat + specs + wiki context	Lấy quyết định, requirement, handoff context

Chọn MCP theo **workflow thật**: code → GitHub, prod incident → Sentry, docs/API → OpenAI Docs hoặc Context7, UI bug → Playwright. Tránh cài “tool zoo” không gắn với task.

STUDIO (local-first)

- Chạy process local, setup nhanh
- Phù hợp filesystem, git, scripts nội bộ
- Dễ dev/test với Inspector
- Trust boundary nhỏ hơn, ít phụ thuộc network

HTTP / Streamable HTTP (shared service)

- Phù hợp SaaS hoặc shared org service
- Hỗ trợ auth, OAuth, centralized rollout
- Nhiều clients dùng chung một endpoint
- Production remote nên ưu tiên hơn SSE legacy

Best Practices với Codex & Claude Code

Dùng MCP như infrastructure, không dùng như “tool zoo”

08

- **Chọn đúng scope:** `local` cho thử nhanh, `project` khi muốn share qua `.mcp.json`, `user` cho default cá nhân nhiều repo
- **Ưu tiên remote HTTP:** dễ OAuth, rollout và policy hơn; chỉ giữ SSE nếu server cũ chưa support transport mới
- **Tận dụng Tool Search:** Claude chỉ nạp tool khi cần, nên server instructions phải nói rõ “khi nào cần search server này”
- **Giữ instructions ngắn và front-load ý chính:** đừng nhét prose dài; phần quan trọng phải xuất hiện sớm
- **Dùng hooks cho guardrail:** match `mcp__server__tool` để log write, block path nguy hiểm, auto-run test sau edit
- **Giảm output size:** trả summary + ID/URI để fetch tiếp, đừng dump JSON lớn làm chậm agent và đốt context

- **Dùng `AGENTS.md` để ép thói quen tốt:** ví dụ “luôn consult OpenAI Docs MCP khi câu hỏi liên quan OpenAI API/Codex”
- **Server label ngắn, mô tả đúng:** tên rõ nghĩa giúp agent chọn đúng server khi có nhiều MCP cùng lúc
- **Thiết kế read path trước write path:** docs/search/fetch trước, mutation sau; review/debug thường không cần write ngay
- **Nếu build remote MCP:** bắt đầu bằng cặp `search + fetch` để agent tìm rồi đọc sâu đúng chỗ
- **Mô tả tool theo kiểu hành động:** thêm “Use this when...”, edge cases, enum, input shape để giảm gọi nhầm tool
- **Khóa phạm vi bằng approval/allowlist:** chỉ mở những tool cần cho task; write actions nhạy cảm nên giữ human approval

Mental model: Codex nên **biết đúng context trước**, rồi mới **hành động** sau.

1. **Ticket** → **code** → **PR**: GitHub MCP + docs MCP. Agent đọc issue/spec, sửa code, mở PR với context đúng.
2. **Prod error** → **root cause**: Sentry MCP + GitHub MCP. Agent lấy stack trace, map sang commit/release, rồi draft fix hoặc issue.
3. **Library upgrade** → **code migration**: Context7/OpenAI Docs MCP + Playwright MCP. Agent đọc doc version mới, refactor, rồi verify UI flow.

Lưu ý: Pattern tốt nhất là pattern có **closed loop**: đọc đúng context → hành động nhỏ → verify ngay. MCP mạnh nhất khi có feedback loop, không chỉ khi có nhiều tools.

1. **Expose remote MCP không auth:** chấp nhận cho internet gọi tool nội bộ
2. **Tin “read-only” tuyệt đối:** prompt injection vẫn có thể biến read path thành data exfiltration path
3. **Nhồi quá nhiều tools vào context:** model tốn token để đọc catalog thay vì giải task
4. **Tool descriptions mơ hồ:** model gọi sai tool dù backend đúng
5. **Không log/audit:** đến lúc có incident thì không biết tool nào đã được gọi
6. **Mix sandbox và production credentials:** local experiment nhưng dùng luôn token prod full quyền

Lưu ý: Trusting MCP developer là **cần**, nhưng vẫn **không đủ**. Cần thêm permissioning, output review, write confirmation, và audit.

- ✓ Bắt đầu bằng workflow thật: ticket → code, dashboard → issue, docs → patch
- ✓ Thiết kế tool names + descriptions để model chọn đúng ngay từ đầu
- ✓ Trả output gọn; để raw payload lớn trong resource hoặc fetch-on-demand
- ✓ Dùng Inspector trước khi test trong client thật
- ✓ Chọn transport đúng: stdio cho local access, HTTP cho shared service
- ✓ Tách read/write scopes, log mọi write action, thêm human confirmation khi cần
- ✓ Với Claude Code/Codex: hướng dẫn agent bằng server naming, instructions, `AGENTS.md`, hooks/policies

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 MCP chuẩn hóa tool integration — build quality MCP servers, **không** build custom adapters per provider
- 2 MCP Inspector là developer essential — test locally **trước khi** LLM integration, tiết kiệm hàng giờ debug
- 3 Resources & Prompts underutilized — dùng cho dynamic context injection, không chỉ mỗi Tools
- 4 Tool description quyết định thành bại — LLM chọn tool 100% dựa trên name + description

Ngày 27: Human-in-the-Loop UX

“Tools connected — tiếp theo thiết kế tương tác người-agent: khi nào agent tự quyết, khi nào cần xin phép?”

- Hoàn thành Lab 26: MCP server + Inspector test
- Đọc: MCP Specification (modelcontextprotocol.io/spec)

Hỏi & Đáp

MCP có thay thế hoàn toàn function calling không? Hay dùng song song?



Cảm ơn!

AICB-P2T3 · Ngày 26 · Model Context Protocol (MCP)

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn